INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS
CAMPUS MONTES CLAROS
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

VITOR TAVARES SIMÕES
MICHAEL DOUGLAS TEIXEIRA BARBOSA

BUSCADOR DE FILMES E CINEMAS

índices, árvore de intervalo e avaliação de desempenho em uma base de 584 mil filmes

MONTES CLAROS
2026

VITOR TAVARES SIMÕES
MICHAEL DOUGLAS TEIXEIRA BARBOSA

BUSCADOR DE FILMES E CINEMAS

índices, árvore de intervalo e avaliação de desempenho em uma base de 584 mil filmes

Relatório apresentado à disciplina Busca e Ordenação do curso de Bacharelado em Ciência da Computação do Instituto Federal do Norte de Minas Gerais, Campus Montes Claros, como parte da avaliação do Projeto 1.

Professor: Tadeu Zubaran

Montes Claros
2026

RESUMO

Este relatório descreve um programa em C++17 que organiza uma base de 584.121 filmes e 400 cinemas para responder consultas por tipo, gênero, duração e ano de lançamento. A solução implementa os Módulos 1 e 2 do enunciado. O Módulo 1 usa listas de posições por categoria, combinadas por interseção e união em tempo linear, e um cache de consultas. O Módulo 2 usa uma árvore binária de busca montada já equilibrada a partir de baldes indexados pelo próprio valor, com busca por intervalo em percurso em ordem com poda. Cada filme é armazenado uma única vez, e todas as estruturas guardam apenas a sua posição no vetor principal. As decisões de projeto foram comparadas por medição com as alternativas descartadas: a árvore é montada em 8,0 ms, contra 28,2 ms de uma árvore AVL; as buscas por intervalo chegam a ser 6.411 vezes mais rápidas que a busca sequencial; e a combinação de filtros caiu de 18,9 s para 1,27 ms no pior caso medido. O carregamento leva cerca de 0,75 s, dos quais 92% correspondem à leitura do arquivo de filmes.

Palavras-chave: busca por intervalo; árvore binária de busca; índice invertido; interseção de listas ordenadas; avaliação de desempenho.

SUMÁRIO

1 INTRODUÇÃO 4
2 BASE DE DADOS E AMBIENTE 4
3 ORGANIZAÇÃO DO PROGRAMA 4
4 MÓDULO 1: BUSCAS CATEGÓRICAS E COMPOSTAS 5
4.1 Índice por categoria 5
4.2 Filtros compostos por intercalação 6
4.3 Cache de consultas 6
5 MÓDULO 2: BUSCAS POR INTERVALO NUMÉRICO 7
5.1 Baldes indexados pelo valor 7
5.2 Montagem da árvore pelo meio 7
5.3 Busca por intervalo com poda 8
5.4 Por que não uma árvore AVL 9
6 BUSCA DE CINEMAS 9
7 AVALIAÇÃO DE DESEMPENHO 9
7.1 Metodologia 9
7.2 Resultados 10
8 VALIDAÇÃO E EXEMPLOS DE EXECUÇÃO 11
9 LIMITAÇÕES 12
10 CONCLUSÃO 12
REFERÊNCIAS 14

1 INTRODUÇÃO

O Projeto 1 da disciplina Busca e Ordenação propõe aplicar os conteúdos de busca e ordenação na organização de uma base de dados de filmes e cinemas (ZUBARAN, 2026). O enunciado exige que o tempo de carregamento e o tempo de cada consulta sejam exibidos e considera preferível um carregamento mais demorado a uma consulta lenta. O trabalho é organizado em módulos opcionais; esta solução implementa o Módulo 1, de buscas categóricas e compostas, e o Módulo 2, de buscas por intervalo numérico.

O programa foi escrito em C++17. Como o enunciado proíbe funções prontas dos assuntos da disciplina, nenhum algoritmo de ordenação ou tabela hash da biblioteca padrão foi usado; as únicas estruturas auxiliares são `std::vector` e `std::string`. A estratégia geral é preparar todas as estruturas uma única vez, ao abrir o programa, para que cada consulta use apenas o que já está pronto.

O código-fonte, as instruções de compilação e a apresentação estão disponíveis no repositório `https://github.com/VitorTavares2/tbo-busca-e-ordenacao`.

As seções seguintes descrevem a base de dados e o ambiente (seção 2), a organização do programa (seção 3), as estruturas de cada módulo (seções 4 a 6), a avaliação de desempenho (seção 7), a validação dos resultados (seção 8), as limitações conhecidas (seção 9) e a conclusão (seção 10).

2 BASE DE DADOS E AMBIENTE

A base é composta por dois arquivos de texto. O arquivo `filmesCrop.txt` tem 48 MB, com um registro por linha e colunas separadas por tabulação, no formato dos conjuntos de dados públicos do IMDb (IMDb, 2026): código, tipo, títulos, ano de início e de fim, duração em minutos e lista de gêneros. Valores ausentes aparecem como `\N`. São 584.121 filmes, dispostos em ordem crescente de código. O arquivo `cinemas.txt` tem 400 cinemas separados por vírgula, cada um com coordenadas, preço do ingresso e uma lista variável de códigos de filmes em cartaz, num total de 2.009 referências.

O programa foi compilado com o g++ 16.1 (MinGW-w64) no Windows 11, com as opções `-O2 -std=c++17 -static`. Todos os tempos foram medidos com `std::chrono::steady_clock`, na mesma máquina e com a mesma base.

3 ORGANIZAÇÃO DO PROGRAMA

O código está dividido em arquivos com responsabilidades separadas, descritos no Quadro 1.

Quadro 1 – Arquivos do programa

Arquivo	**Responsabilidade**
`modelos.h`	Estruturas `Filme` e `Cinema`.
`texto.cpp`	Funções de apoio: dividir uma linha, remover espaços, converter número e passar para minúsculas.
`entrada.cpp`	Leitura do teclado com validação dos valores digitados.
`carregador.cpp`	Leitura dos dois arquivos e ligação de cada cinema aos filmes que exibe.
`indice.cpp`	Módulo 1: índice por categoria, interseção e união.
`arvore.cpp`	Módulo 2: baldes, montagem da árvore e busca por intervalo.
`cache.cpp`	Cache de consultas.
`main.cpp`	Montagem das estruturas, menu e as cinco buscas.

Fonte: elaborado pelos autores (2026).

Ao iniciar, o programa lê os filmes e os cinemas, liga cada cinema às posições dos filmes que exibe e percorre o vetor de filmes uma única vez para preencher, ao mesmo tempo, os dois índices do Módulo 1 (tipo e gênero) e os baldes do Módulo 2 (ano e duração). Depois desse laço, as duas árvores são montadas, o tempo de carregamento é exibido e o programa entra no menu, que oferece cinco buscas: filmes por tipo ou gênero, cinemas que exibem filmes por tipo ou gênero, filmes por duração, filmes por ano e cinemas que exibem filmes de um ano ou período.

A decisão central do projeto é armazenar cada filme uma única vez, no vetor principal. Índices, árvores e cinemas guardam apenas a posição do filme nesse vetor, um número inteiro. Isso reduz a memória usada, evita cópias de texto e atende, como consequência, ao requisito de evitar *aliasing* descrito no Módulo 5: um mesmo filme é ordenado logicamente por vários critérios sem ser duplicado fisicamente.

4 MÓDULO 1: BUSCAS CATEGÓRICAS E COMPOSTAS

4.1 Índice por categoria

A classe `IndiceCategoria` associa cada nome de categoria a uma lista com as posições dos filmes daquela categoria, estrutura conhecida como índice invertido. O programa mantém dois índices: um por tipo, com 10 valores distintos, e outro por gênero, com 27 valores. Um filme com vários gêneros entra na lista de cada um deles.

Como os filmes são percorridos na ordem do vetor, as posições entram em ordem crescente e toda lista já fica ordenada, sem nenhuma etapa de ordenação. Essa propriedade é a base das operações da subseção 4.2. A busca de uma categoria percorre a lista de nomes, que

tem no máximo 27 elementos, e compara os textos sem diferenciar letras maiúsculas e minúsculas. A construção dos dois índices leva 44,2 ms.

4.2 Filtros compostos por intercalação

A função `aplicarFiltros` obtém a lista de cada filtro e a combina com o resultado acumulado: o operador E usa a interseção, e o operador OU usa a união. As duas operações percorrem as listas ordenadas ao mesmo tempo, avançando sempre o índice do menor elemento, como na etapa de intercalação do *merge sort* (CORMEN et al., 2012). O Código 1 mostra a interseção.

Código 1 – Interseção de duas listas ordenadas (`indice.cpp`)

```
std::vector<int> intersecao(const std::vector<int>& a,
                            const std::vector<int>& b) {
    std::vector<int> resultado;
    size_t i = 0;
    size_t j = 0;
    while (i < a.size() && j < b.size()) {
        if (a[i] == b[j]) {
            resultado.push_back(a[i]);
            i++;
            j++;
        } else if (a[i] < b[j]) {
            i++;
        } else {
            j++;
        }
    }
    return resultado;
}
```

Fonte: elaborado pelos autores (2026).

Se `a[i]` é menor que `b[j]`, ele não pode aparecer no restante de `b`, que só tem valores maiores, e pode ser descartado sem nova busca. Cada passo avança pelo menos um índice, de modo que o custo é O(|a| + |b|). A união segue o mesmo esquema, incluindo uma vez os elementos comuns e copiando ao final o que resta de cada lista. Como o resultado continua ordenado, três ou mais filtros podem ser encadeados. A versão anterior procurava cada elemento de uma lista dentro da outra inteira, com custo O(|a| · |b|); a comparação está na subseção 7.2.

4.3 Cache de consultas

A classe `CacheConsultas` guarda o resultado de cada busca associado a uma chave que descreve a consulta (alvo, operador e filtros, com os valores em minúsculas). Antes de calcular uma busca, o programa consulta o cache e, se a chave já existir, devolve o resultado

guardado. Na consulta `tvEpisode E Drama`, o tempo cai de 1,27 ms para 0,017 ms na repetição.

A localização da chave é feita por comparação sequencial entre as chaves guardadas, com custo O(B), em que B é o número de consultas já feitas na sessão. O enunciado pede O(1), o que exigiria uma tabela hash implementada do zero; essa diferença é tratada na seção 9.

5 MÓDULO 2: BUSCAS POR INTERVALO NUMÉRICO

O Módulo 2 pede que ano e duração sejam indexados em uma árvore, para que a busca por intervalo seja mais rápida que a busca sequencial. A classe `ArvoreNumerica` é usada duas vezes, uma para o ano e outra para a duração, e é construída em três passos: distribuição em baldes, montagem pelo meio e busca com poda.

5.1 Baldes indexados pelo valor

Durante o laço de carregamento, cada filme é colocado em `filmesPorValor[valor]`: o índice do vetor é o próprio valor numérico. A inserção custa O(1) e não faz nenhuma comparação; filmes sem ano ou sem duração (valor −1) ficam de fora. Ao final, os nós da árvore são criados percorrendo o vetor de baldes do índice 0 ao maior valor e aproveitando apenas os baldes não vazios. Como o índice cresce, os valores distintos já saem em ordem crescente, a mesma ideia da ordenação por contagem (CORMEN et al., 2012). Esse passo custa O(V + K), em que V é o maior valor mais um e K é o número de valores distintos. Na duração, V = 51.421 e K = 372; no ano, K = 134.

5.2 Montagem da árvore pelo meio

Com os valores distintos em ordem, a função recursiva `ligar`, mostrada no Código 2, escolhe o elemento do meio de cada trecho como raiz, liga a metade menor à esquerda e a metade maior à direita.

Código 2 – Montagem da árvore pelo meio (`arvore.cpp`)

```
int ArvoreNumerica::ligar(int inicio, int fim) {
    if (inicio > fim) {
        return -1;
    }
    int meio = (inicio + fim) / 2;
    nos[meio].esquerda = ligar(inicio, meio - 1);
    nos[meio].direita = ligar(meio + 1, fim);
    return meio;
}
```

Fonte: elaborado pelos autores (2026).

Um trecho com `inicio > fim` está vazio e indica a ausência de filho. Cada chamada recebe um trecho com menos da metade dos elementos do anterior, o que garante o término da recursão. Como as duas metades diferem em no máximo um elemento, a árvore resultante tem a menor altura possível, $\lceil \log_2(K + 1) \rceil$: 8 níveis para os anos e 9 para as durações. A montagem visita cada nó uma vez, com custo O(K).

5.3 Busca por intervalo com poda

A busca por intervalo usa a função `coletar` (Código 3), um percurso em ordem que só desce para a esquerda se o valor do nó for maior que o mínimo e só desce para a direita se for menor que o máximo. Quando o valor está dentro do intervalo, o balde inteiro é copiado para o resultado.

Código 3 – Busca por intervalo (`arvore.cpp`)

```
void ArvoreNumerica::coletar(int no, int minimo, int maximo,
                             std::vector<int>& resultado) const {
    if (no < 0) {
        return;
    }
    const No& atual = nos[no];
    if (atual.valor > minimo) {
        coletar(atual.esquerda, minimo, maximo, resultado);
    }
    if (atual.valor >= minimo && atual.valor <= maximo) {
        const std::vector<int>& filmes = filmesPorValor[atual.valor];
        resultado.insert(resultado.end(), filmes.begin(), filmes.end());
    }
    if (atual.valor < maximo) {
        coletar(atual.direita, minimo, maximo, resultado);
    }
}
```

Fonte: elaborado pelos autores (2026).

As subárvores que não podem conter valores do intervalo são descartadas sem serem visitadas. O custo é proporcional aos nós visitados mais a quantidade R de filmes copiados; na

busca pelo ano 2000, apenas 7 dos 134 nós são visitados. Como o percurso é em ordem, o resultado sai organizado por valor crescente, sem ordenação posterior.

5.4 Por que não uma árvore AVL

A versão anterior do programa usava uma árvore AVL, que mantém o equilíbrio a cada inserção por meio de rotações (ADELSON-VELSKY; LANDIS, 1962). Esse mecanismo é necessário quando os dados mudam durante o uso. Neste projeto, nenhum filme é inserido ou removido depois do carregamento, e todos os valores são conhecidos antes da montagem. Montar a árvore uma vez pelo meio produz o mesmo equilíbrio sem rotações e sem recalcular alturas. Na prática, a árvore atual ficou mais baixa (8 e 9 níveis, contra 9 e 10 da AVL) e foi montada em 8,0 ms, contra 28,2 ms da AVL e 36,3 ms de uma alternativa com *merge sort* (Tabela 2).

6 BUSCA DE CINEMAS

Cada cinema traz os códigos dos filmes que exibe. Durante o carregamento, a função `resolverFilmesDosCinemas` troca cada código pela posição do filme no vetor, usando busca binária, já que os filmes estão em ordem crescente de código (SEDGEWICK; WAYNE, 2011). O programa confere essa ordem antes e, se ela não valer, usa busca sequencial. As 2.009 referências são resolvidas em 4,8 ms, contra 1.828 ms e 374,8 milhões de comparações da busca sequencial. Códigos inexistentes ficam com posição −1 e são ignorados nas buscas.

Na busca de cinemas, os filmes encontrados pelo índice ou pela árvore são marcados em um vetor de N posições. Em seguida, cada cinema é percorrido, e basta acessar a posição de cada um de seus filmes nesse vetor para saber se ele foi encontrado; a verificação de um cinema termina no primeiro filme marcado. O custo é O(N + R + F), em que F é o total de referências dos cinemas. Mesmo com 559 mil filmes marcados, a busca leva 0,25 ms e retorna 391 cinemas.

7 AVALIAÇÃO DE DESEMPENHO

7.1 Metodologia

O próprio programa mede e exibe o tempo de carregamento e o tempo de cada busca. Para comparar as escolhas de projeto com as alternativas descartadas, foi usado um programa de medição separado, que chama as mesmas funções do projeto. As alternativas (a árvore AVL da versão anterior, um *merge sort* implementado à mão, a busca sequencial e a versão anterior

da interseção e da união) foram programadas e devolvem os mesmos resultados da solução final.

Em cada medição, os dados são carregados antes do cronômetro. A operação é executada várias vezes seguidas, e o tempo total é dividido pelo número de repetições; operações mais rápidas são repetidas mais vezes, para que o cronômetro tenha precisão suficiente. O conjunto completo de medições foi executado três vezes, e cada tabela apresenta a mediana das três execuções.

7.2 Resultados

A Tabela 1 detalha o tempo de carregamento. A leitura e a interpretação do arquivo de filmes representam 92% do total e são inevitáveis em qualquer solução, pois os 48 MB precisam ser lidos uma vez. As estruturas criadas por escolha do projeto somam cerca de 57 ms, valor pago uma única vez.

Tabela 1 – Tempo de carregamento por etapa

Etapa	**Tempo (ms)**	**Participação**
Ler e interpretar os filmes	710,1	92,4%
Montar os índices de tipo e gênero	44,2	5,8%
Árvores de ano e duração (baldes e montagem)	7,6	1,0%
Ligar cinemas aos filmes	5,7	0,7%
Ler os cinemas	1,0	0,1%
Total	**768,6**	**100%**

Fonte: elaborado pelos autores (2026).

A Tabela 2 compara três formas de montar as estruturas de ano e duração com os 750 mil valores válidos.

Tabela 2 – Montagem das estruturas de ano e duração

Alternativa	**Tempo (ms)**	**Altura (ano / duração)**
Baldes e montagem pelo meio (adotada)	8,0	8 / 9
Árvore AVL (versão anterior)	28,2	9 / 10
Merge sort e montagem pelo meio	36,3	8 / 9

Fonte: elaborado pelos autores (2026).

A Tabela 3 compara a busca por intervalo na árvore com a busca sequencial, que examina os 584.121 filmes. A vantagem é maior quando o resultado é pequeno; quando o intervalo cobre quase toda a base, o tempo passa a ser dominado pela cópia dos 559 mil resultados, e a vantagem cai para 5,1 vezes. O custo da árvore acompanha o tamanho da resposta, e não o tamanho da base.

Tabela 3 – Busca por intervalo: árvore e busca sequencial

Consulta	Filmes	Nós visitados	Árvore (ms)	Sequencial (ms)	Ganho
Ano 2000	5.192	7 de 134	0,0006	3,67	6.411×
Duração de 90 a 120 min	8.501	39 de 372	0,0036	2,57	711×
Anos de 1990 a 1999	57.239	16 de 134	0,025	3,46	136×
Duração de 1 a 59 min	153.039	64 de 372	0,074	4,48	61×
Anos de 1885 a 2026	559.303	134 de 134	0,98	5,02	5,1×

Fonte: elaborado pelos autores (2026).

A Tabela 4 mostra o efeito da intercalação nos filtros compostos. A versão anterior era mais lenta até do que examinar a base inteira sem índice; na consulta `tvEpisode E Drama`, que cruza listas de 474.707 e 191.271 posições, a versão atual é cerca de 14.900 vezes mais rápida.

Tabela 4 – Filtros compostos

Consulta	Filmes	Versão anterior (ms)	Sem índice (ms)	Atual (ms)
short E Animation	2.080	370,8	6,2	0,13
movie E Comedy	3.382	737,2	6,1	0,25
tvEpisode E Drama	161.744	18.900	14,4	1,27
Comedy OU Drama	287.628	5.600	–	1,27

Fonte: elaborado pelos autores (2026).

Por fim, a Tabela 5 reúne as demais medições citadas nas seções 4 e 6.

Tabela 5 – Outras medições

Operação	Adotada (ms)	Alternativa (ms)
Ligar 2.009 referências de cinema a filmes (busca binária × sequencial)	4,8	1.828
Repetir `tvEpisode E Drama` (cache × recalcular)	0,017	1,27
Achar os cinemas de 559 mil filmes (vetor de marcação)	0,25	–

Fonte: elaborado pelos autores (2026).

8 VALIDAÇÃO E EXEMPLOS DE EXECUÇÃO

Os totais devolvidos pelo programa foram conferidos por contagem direta nos arquivos de dados, de forma independente do código. A Tabela 6 apresenta exemplos de consultas e os resultados obtidos. Os tipos e gêneros são aceitos sem diferenciar maiúsculas e minúsculas.

Tabela 6 – Exemplos de consultas

Opção	Consulta	Resultado
1	tipo `movie` E gênero `Comedy`	3.382 filmes
1	gênero `Comedy` OU gênero `Drama`	287.628 filmes
1	gênero `comedy` (em minúsculas)	125.933 filmes
2	cinemas que exibem filmes do gênero `Documentary`	96 cinemas
3	duração de 90 a 120 minutos	8.501 filmes
4	ano 2000	5.192 filmes
4	anos de 1990 a 1999	57.239 filmes
5	cinemas que exibem filmes do ano 2000	24 cinemas

Fonte: elaborado pelos autores (2026).

O programa também trata entradas inválidas sem encerrar a execução: texto onde se espera número, opção fora do menu, campo diferente de tipo ou gênero, operador diferente de E ou OU, valor mínimo maior que o máximo e categoria inexistente. Em cada caso, a mensagem explica o problema e a pergunta é repetida; buscas sem resultado exibem uma dica, como o intervalo de anos disponível na base (1885 a 2026).

9 LIMITAÇÕES

- Os Módulos 3, 4 e 5 não foram implementados. Do Módulo 5, apenas o requisito de evitar *aliasing* é atendido, como consequência da organização por posições.
- O cache localiza as chaves por comparação sequencial, com custo O(B), e não O(1) como pede o enunciado. Uma tabela hash própria resolveria esse ponto; com as poucas consultas de uma sessão, o custo medido é desprezível.
- A árvore é estática: incluir filmes depois do carregamento exigiria montá-la novamente. Para dados que mudam durante o uso, uma árvore autobalanceada, como a AVL, seria mais adequada.
- O vetor de baldes tem o tamanho do maior valor encontrado (51.421 posições na duração), e não do número de valores distintos.

10 CONCLUSÃO

A solução implementa os Módulos 1 e 2 seguindo a orientação do enunciado de concentrar o custo no carregamento. O programa abre em cerca de 0,75 s, dos quais a maior parte é a leitura obrigatória do arquivo, e responde às buscas em frações de milissegundo.

As principais decisões foram guardar cada filme uma única vez e usar sua posição em todas as estruturas, manter as listas do índice em ordem para combiná-las por intercalação e,

no Módulo 2, aproveitar o fato de os dados não mudarem para montar a árvore já equilibrada a partir de baldes. As medições confirmaram cada escolha frente às alternativas: a montagem ficou 3,5 vezes mais rápida que a da AVL, a busca por intervalo superou a busca sequencial em todos os casos medidos e a combinação de filtros deixou de ser o ponto mais lento do programa.

REFERÊNCIAS

ADELSON-VELSKY, G. M.; LANDIS, E. M. An algorithm for the organization of information. **Soviet Mathematics Doklady**, [*s. l.*], v. 3, p. 1259-1263, 1962.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos**: teoria e prática. Tradução da 3. ed. americana. Rio de Janeiro: Elsevier, 2012.

IMDb. **IMDb non-commercial datasets**. [*S. l.*], 2026. Disponível em: https://developer.imdb.com/non-commercial-datasets/. Acesso em: 20 out. 2026.

SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Upper Saddle River: Addison-Wesley, 2011.

ZUBARAN, T. **Busca e Ordenação**: Projeto 1. Montes Claros: IFNMG, 2026. Enunciado do trabalho da disciplina.